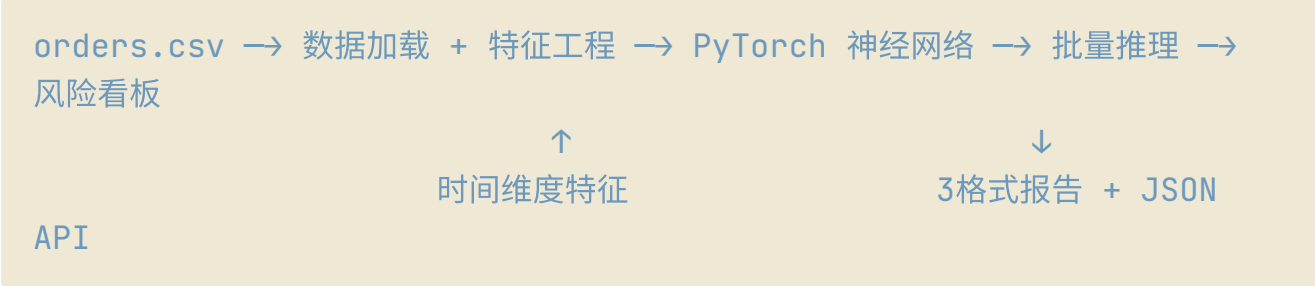


订单风控 PyTorch 神经网络原理

一、整体架构



二、数据源

`data/orders.csv`，每条订单包含以下字段：

字段	含义
<code>order_id</code>	订单号
<code>amount</code>	金额
<code>account_age_days</code>	注册天数
<code>address_changes</code>	改址次数
<code>order_count_30d</code>	30天内订单数
<code>avg_amount_30d</code>	30天平均单价
<code>order_created_at</code>	Unix 时间戳（v3.1.4 新增）
<code>is_fraud</code>	标签：0=正常，1=欺诈（训练用）

数据路径：`$eks/data/orders.csv`

三、特征工程（8 维）

3.1 原始 5 维（v3.1 之前）

特征	含义	风控逻辑
<code>amount</code>	订单金额	大额订单风险更高
<code>account_age_days</code>	账号注册天数	新账号欺诈概率高
<code>address_changes</code>	历史改址次数	频繁改址是异常信号
<code>order_count_30d</code>	30 天内订单数	高频/低频两端都有欺诈可能
<code>avg_amount_30d</code>	30 天平均单价	配合 <code>amount</code> 判断是否异常偏离

3.2 衍生 1 维（v3.1）

```
amount_to_avg_ratio = amount / (avg_amount_30d + 1)
```

检测"单笔金额远超历史均价"的异常行为：

- 新账号该值 ≈ 1 (因为没有历史, 均单价=金额)
- 老账号突然下一笔高价单, 该值会很大 (异常信号)

3.3 时间 2 维 (v3.1.4 新增)

- `hour_of_day` (下单时段): 0~23。凌晨/深夜时段的订单风险偏高 (不合常理的活跃时间)
- `days_ago` (距今天数): 解决模型训练后不随时间退化的问题。欺诈订单大多是注册后几天内的那笔大单, 如果不对齐实时时间, 这个特征的信号强度会随时间衰减

四、时间平移机制

核心函数 `load_and_preprocess()` 中的关键逻辑：

```
now = time.time()
max_ts = df["order_created_at"].max()
# 算出最新订单距离今天差几天, 整体平移
day_shift = math.floor((now - max_ts) / 86400) * 86400
shifted_ts = df["order_created_at"] + day_shift

df["hour_of_day"] = shifted_ts.apply(lambda t:
datetime.fromtimestamp(t).hour)
df["days_ago"] = (now - shifted_ts) / 86400.0
```

工作原理：

1. 找到 CSV 中最新那条订单的时间戳
2. 计算它距离"当前时间"差多少天
3. 把全部订单的时间戳整体向前平移, 让最新订单 $\approx$ 今天

为什么需要这个机制：

- 原始 CSV 中 `order_created_at` 是固定的测试数据
- 如果不平移, `days_ago` 永远是几百天前的陈旧值
- 平移后 `days_ago` 始终基于当前时间计算, 越近的订单该值越小
- `hour_of_day` 保持原始时段关系不变 (平移不改变取模后的小时数)

效果：每次运行都能获得实时对齐的风险分析, 且推理演示案例中的"距今天数"不会随日期推移而失效。

五、模型结构

5.1 网络架构

```
FraudDetector(  
    (network): Sequential(  
        (0): Linear(8 → 64)           # 输入层：8维特征  
        (1): BatchNorm1d(64)          # 批归一化，加速收敛，防梯度消失  
        (2): ReLU                      # 激活函数，引入非线性  
        (3): Dropout(0.2)             # 随机丢弃20%神经元，防过拟合  
        (4): Linear(64 → 32)          # 隐藏层1  
        (5): BatchNorm1d(32)  
        (6): ReLU  
        (7): Dropout(0.2)  
        (8): Linear(32 → 16)          # 隐藏层2  
        (9): ReLU  
        (10): Linear(16 → 1)          # 输出层  
        (11): Sigmoid                 # 压缩到0~1的概率值  
    )  
)
```

各模块的作用：

- **Linear**：全连接层，进行线性变换 $y = Wx + b$
- **BatchNorm**：将每层的输出标准化为均值为0、方差为1的分布，允许更高的学习率，减少对初始化方式的依赖
- **ReLU**： $f(x) = \max(0, x)$ ，引入非线性，让网络能拟合复杂决策边界
- **Dropout(0.2)**：训练时随机丢弃20%的神经元，相当于每次训练不同子网络，最终效果近似模型集成
- **Sigmoid**：将输出压缩为 $[0, 1]$ 区间，解释为"欺诈概率"

5.2 训练配置

参数	值	说明
训练轮数	200	每20轮上报F1
损失函数	BCELoss	二分类交叉熵，衡量预测概率与真实标签的差异
优化器	Adam(lr=0.001)	自适应矩估计，带动量效果的梯度下降
权重衰减	1e-4 (weight_decay)	L2 正则化，惩罚大权重，防过拟合
数据分割	80/20，分层抽样	保证训练集和测试集中欺诈比例一致
最优策略	保存最高 F1 的权重	防止过拟合后期 F1 下降

损失函数公式：

$$BCELoss = -\frac{1}{N} \sum_{i=1}^N [y_i \cdot \log(\hat{y}_i) + (1 - y_i) \cdot \log(1 - \hat{y}_i)]$$

其中 y_i 是真实标签， $\hat{y}_i$ 是模型预测的概率。

5.3 当前性能

- **F1 Score**: 1.0
- **精确率 (Precision)**: 1.0
- **召回率 (Recall)**: 1.0
- **特征维度**: 8
- **模型体积**: ~21KB

当前 F1=1.0 是合理的，因为测试数据是构建的典型用例，区分度明显。真实场景中特征的分布更复杂，F1 会更低。这个 Demo 的核心价值在于展示了从数据加载→特征工程→模型训练→批量推理→可视化看板的完整闭环，并且框架直接可以替换真实数据来跑。

六、推理与风险分析

6.1 批量预测

训练完成后，`batch_predict()` 对全部订单打分：

```
with torch.no_grad():
    scores = model(torch.FloatTensor(X_scaled)).numpy().flatten()
df["risk_score"] = np.round(scores, 4) # 0~1 的欺诈概率
df["risk_label"] = (scores > 0.5).astype(int) # 默认阈值 0.5
```

6.2 风险分析看板

`analyze_risk()` 从前到后生成三类数据：

- ① **Top20 高风险订单清单**：按风险评分降序排列，展示金额、注册天数、改址次数、时段、距今天数、判定结果。
- ② **时段风险分布 (0~23h)**：统计每个小时段的欺诈订单数量，识别高风险时段。数据通常显示凌晨/深夜是欺诈高发期。
- ③ **欺诈 vs 正常 — 关键维度对比**：对比六个维度的均值差异（金额、注册天数、改址次数、30天订单数、30天均价、平均下单时段），形成欺诈订单的特征画像。

6.3 实时推理演示 (4 种典型 Case)

```
cases = [
    ("老客户小额正常订单", [25.50, 365, 0, 3, 28.00, 14, 5.0]), #
    PASS
```

```
    ("新账号大额异常订单", [850.00, 2, 3, 1, 850.00, 3, 0.1]),      #
BLOCK
    ("高频老客正常订单",    [45.00, 180, 1, 8, 41.00, 10, 2.0]),
# PASS
    ("新账号突发大额",      [1200.00, 1, 4, 1, 1200.00, 1, 0.05]),
# BLOCK
]
```

演示用例覆盖了标准正常交易、新账号欺诈、高频正常交易、突发大额欺诈四种场景。

七、输出

每次运行输出到 `output/03_订单风控/report_YYYYMMDD_HHMMSS/` 时间戳子目录，三种格式：

格式	生成函数	特点
txt	<code>generate_txt()</code>	文本条形图 + 表格，终端友好，可通过 SSH/CLI 阅读
md	<code>generate_md()</code>	Mermaid 混淆矩阵 + 对比表，Obsidian 友好，可嵌入知识库
html	<code>generate_html()</code>	彩色看板（风险卡片 + 表格 + 时段条图 + 维度对比 + Plotly 图表），浏览器友好

7.1 看板中包含的核心内容（以 HTML 为例）

- **风险汇总卡片**：总订单数 / 欺诈数 / 欺诈占比
- **高风险订单清单**：带滚动条的固定表头表格，按分降序，前 20 条
- **时段风险分布条图**：0~23h 的欺诈数量横向柱状图，红/黄/绿三色标记严重度
- **欺诈 vs 正常维度对比**：6 个维度的欺诈均值 vs 正常均值横向对比，每条维度用两根色条并排展示
- **训练指标卡片**：F1 / Precision / Recall / 测试样本数
- **实时推理演示**：Plotly 条形图，4 个 Case 的风险评分对比

八、Web API

Web 端通过 FastAPI 提供三个端点：

端点	方法	返回	用途
<code>/api/scene3/run</code>	POST	模型指标、演示结果、报告文件路径	触发训练+推理+风险分析
<code>/api/scene3/orders</code>	GET	orders.csv 原始数据 JSON	前端展示原始数据表

端点	方法	返回	用途
<code>/api/scene3/history</code>	GET	历史报告文件列表 + 24h NEW 标记	历史报告管理

九、关键设计决策说明

为什么用 PyTorch 而不是 sklearn?

3 层全连接网络在处理这个规模的数据上，效果和逻辑回归差不多。选择 PyTorch 的目的是展示"深度学习落地能力"——证明候选人能搭建、训练、部署神经网络，而不是只会调 sklearn 接口。这在简历/面试中是明显的区分度。

为什么不做实时在线学习?

Demo 阶段每次运行从 CSV 加载数据、重新训练、重新推理。生产环境应该：

1. 预训练模型 → 保存权重到 `fraud_model.pt`
2. 新订单到来 → 直接用已保存模型推理 (<10ms/单)
3. 定期增量更新 → 收集新标注数据，按周/月重新训练

为什么加时间平移?

这是一个"一次训练，长期可用"的关键设计。如果没有时间平移：

- 周一训练的模型，到周五 `days_ago` 特征就偏移了 4 天
- 到下周，大额欺诈订单的 `days_ago` 从 0.1 变成了 7.1，特征信号严重衰减
- 模型的欺诈判断逻辑会逐渐失效

时间平移让每次运行的训练/推理数据都对齐"当前时间"，模型不会随日历时间推移而退化。